

2.10 - EDA: os desafios ocultos

Ordenação, idempotência e rastreabilidade

Os três desafios que a EDA esconde

A conta que chega em produção

- **Eventos chegam fora da ordem em que ocorreram**
 - Sequência não é garantida por padrão no broker
- **O mesmo evento pode chegar mais de uma vez**
 - At-least-once delivery é o comportamento comum
- **Rastrear um evento entre consumers é difícil**
 - Sem correlação de IDs, o caminho se perde
- **Falha no consumer não pode perder o evento**
 - Sem rede de segurança, o fato simplesmente desaparece

Consumer idempotente por chave de evento

JAVASCRIPT

```
// Processa cada evento uma unica vez, mesmo duplicado
async function consumir(evento) {
  const jaProcessado = await store.existe(evento.id);
  if (jaProcessado) return; // ignora a duplicata

  await aplicar(evento);
  await store.marcar(evento.id);
}
```

Dead-letter queue como rede de segurança

- **Evento que falha repetidas vezes vai para a DLQ**
 - O fluxo principal segue sem travar nos erros
- **A DLQ guarda o que precisa de atenção humana**
 - Nada se perde, tudo fica disponível para reprocessar
- **Correlação de IDs reconstrói o caminho do evento**
 - Cada salto carrega o mesmo trace para auditoria
- **Ordenação garantida só quando o domínio exige**
 - Forçar ordem global custa caro em escala

At-least-once exige idempotência

- Entrega garantida quase sempre significa entrega repetida
- Consumer não idempotente vira prejuízo financeiro

Se o broker entrega pelo menos uma vez, o mesmo evento vai chegar duas vezes em algum momento. Sem idempotência no consumer, isso vira cobrança em dobro, estoque errado ou pior.

- Idempotência não é opcional, é requisito de arquitetura

Pagamento processado duas vezes

- **Evento de entrega confirmada chegou duas vezes**
 - O consumer de billing não era idempotente

LUNALOG

Uma entrega da LunaLog teve o pagamento processado duas vezes no mesmo dia. O evento de entrega confirmada foi entregue duas vezes pelo broker, situação normal em at-least-once delivery, e o consumer do billing não era idempotente. O crédito em dobro virou prejuízo real e uma ligação furiosa do cliente às oito da manhã. Naquele dia, idempotência deixou de ser conceito de slide e virou requisito de arquitetura com custo medido em reais.



Eventos resolvem a comunicação entre domínios, e você já sabe domar o caos deles. O próximo passo não é sobre como os serviços conversam, é sobre como eles são empacotados e operados. Contêineres parecem só uma forma de deploy, mas mudam o próprio design dos serviços. Por que um arquivo salvo no disco pode quebrar todo o seu sistema?

Contêineres: design stateless e imagem imutável